JÖNKÖPING UNIVERSITY

Dynamic SVG & JS

Jérôme LANDRÉ

2024-03-25

Outline

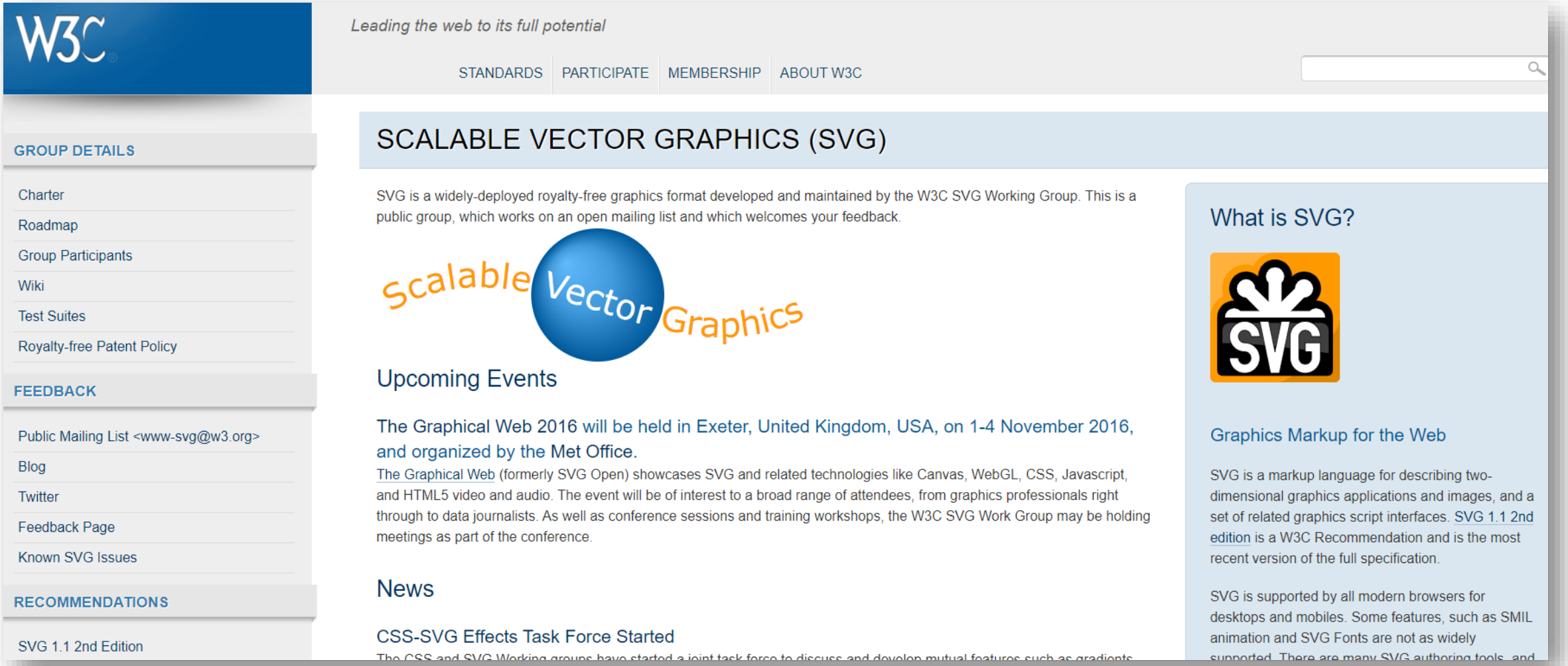
JÖNKÖPING UNIVERSITY

- 1) Scalable Vector Graphics (SVG)
- 2) Bitmap graphics
- 3) SVG and Adobe Illustrator
- 4) SVG basics
- 5) SVG and Javascript
- 6) SVG Events
- 7) Resources

1) Scalable Vector Graphics (SVG)

JÖNKÖPING UNIVERSITY

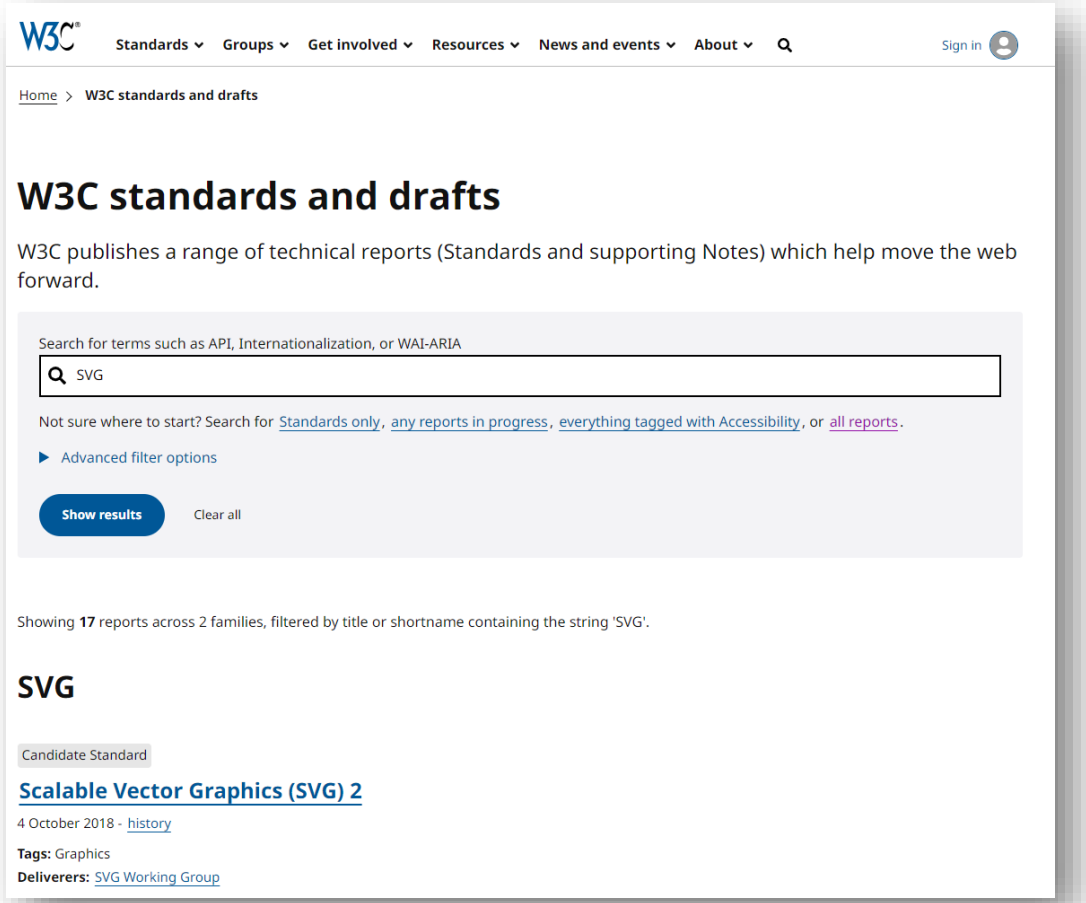
→ SVG is an **open standard XML-based file format** for describing **vector graphics**

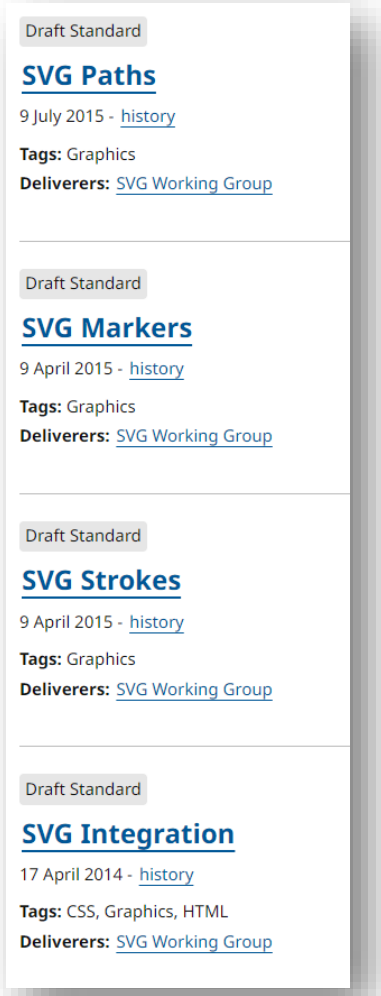


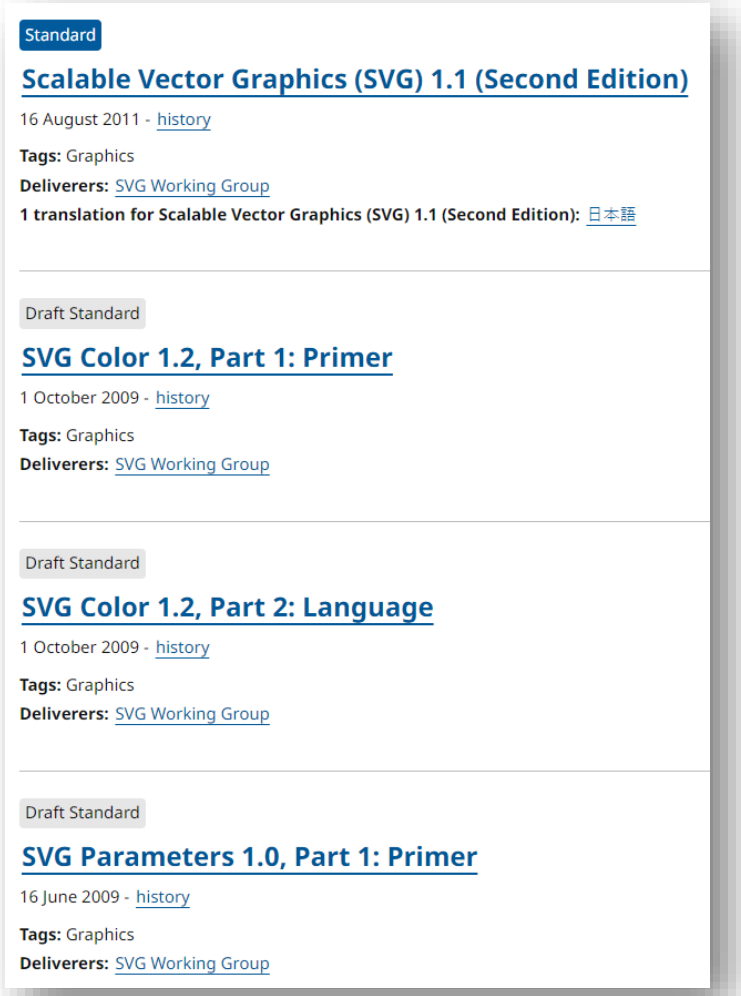
1) Scalable Vector Graphics (SVG)

JÖNKÖPING UNIVERSITY

→ SVG is an **open standard** **XML-based file format** for describing **vector graphics**







1) Scalable Vector Graphics (SVG)

JÖNKÖPING UNIVERSITY

- → SVG is an **open standard XML-based file format** for describing **vector graphics**
- → A ".svg" file is a **text file**
- → XML is a tag language (like HTML) in which you describe the mathematical objects you want to create with **tags, properties and values**




```
1  <svg version="1.1"
2      width="300" height="200"
3      xmlns="http://www.w3.org/2000/svg">
4
5    <rect width="100%" height="100%" fill="red" />
6
7    <circle cx="150" cy="100" r="80" fill="green" />
8
9    <text x="150" y="125" font-size="60"
10     text-anchor="middle" fill="white">SVG</text>
11
12 </svg>
```

Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Getting_Started

1) Scalable Vector Graphics (SVG)

JÖNKÖPING UNIVERSITY

→ XML is a tree description language: please, draw the **tree** of this program

```svg
<svg version="1.1"
     width="300" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <rect width="100%" height="100%" fill="red" />

  <circle cx="150" cy="100" r="80" fill="green" />

  <text x="150" y="125" font-size="60"
    text-anchor="middle" fill="white">SVG</text>

</svg>
```

Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Getting_Started

1) Scalable Vector Graphics (SVG)

JÖNKÖPING UNIVERSITY

- → SVG is an **open standard XML-based file** format for describing **vector graphics**
- → Representing graphics as math objects (vectors, curves) => zoom in and out can be applied without any loss of information



x 3 =

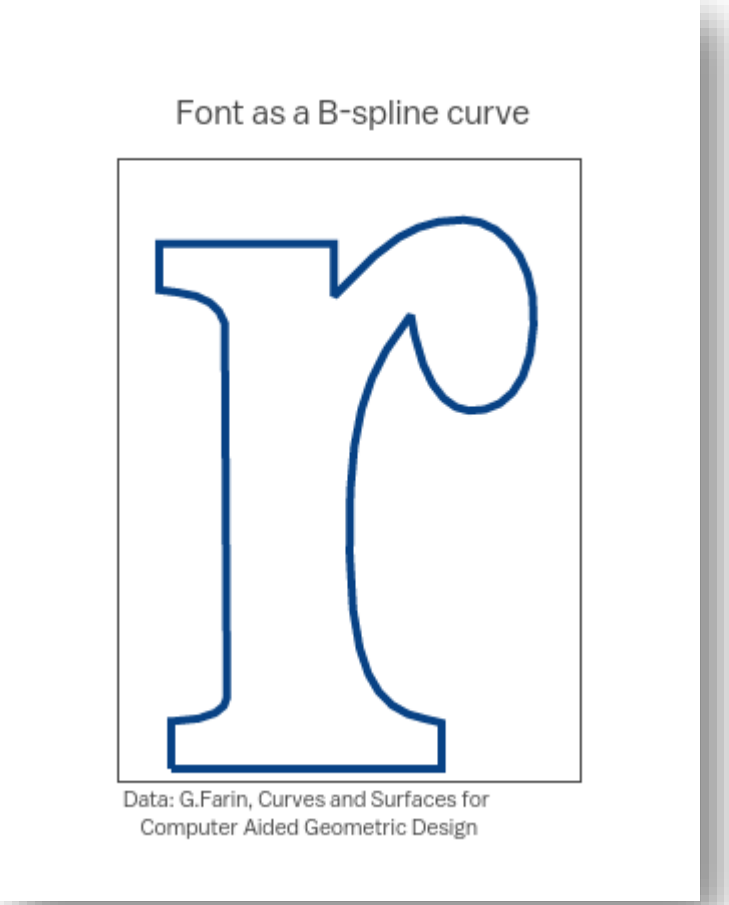
Why?

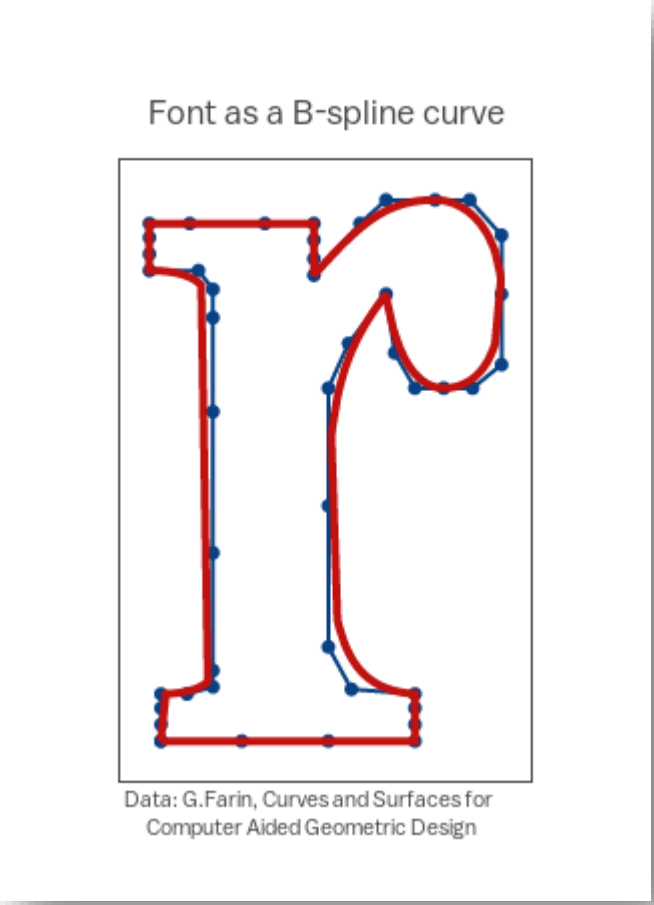


1) Scalable Vector Graphics (SVG)

JÖNKÖPING UNIVERSITY

- → SVG is an open standard **XML-based file** format for describing **vector graphics**
- → Representing graphics as **math objects (vectors, curves)** => zoom in and out can be applied without any loss of information



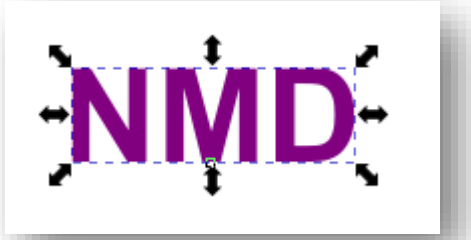


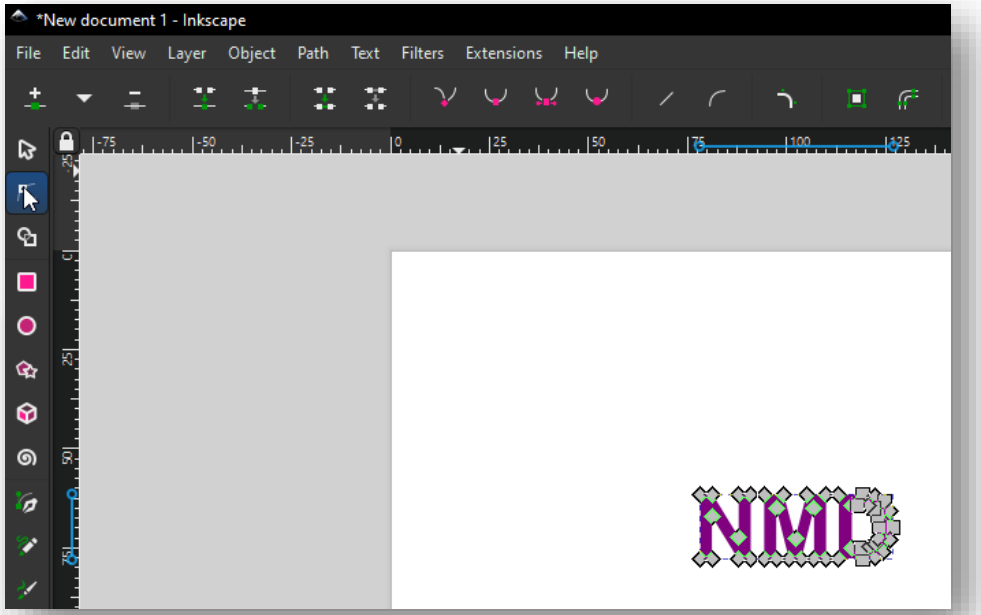
Source: https://chart-studio.plotly.com/~empet/279.embed

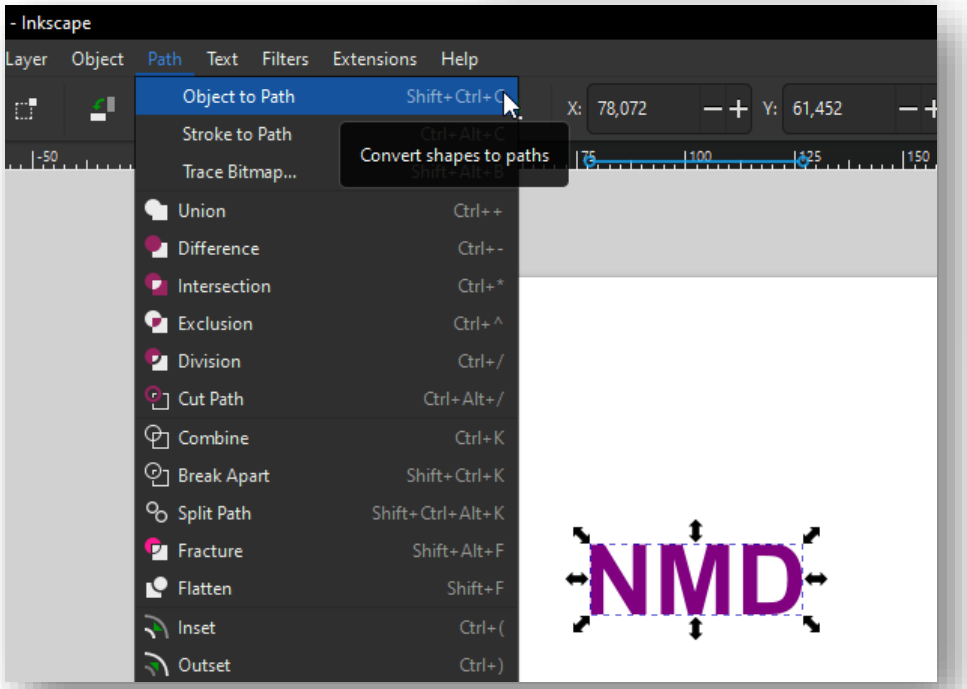
1) Scalable Vector Graphics (SVG)

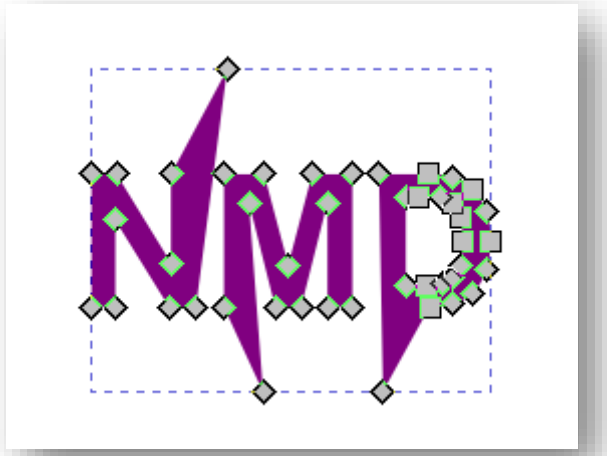
JÖNKÖPING UNIVERSITY

NMD











2) Bitmap graphics

JÖNKÖPING UNIVERSITY

→ Bitmap graphics is used to encode images as a 2D array of pixels, so you cannot zoom in a lot!



320x200

x 2 =



640x400

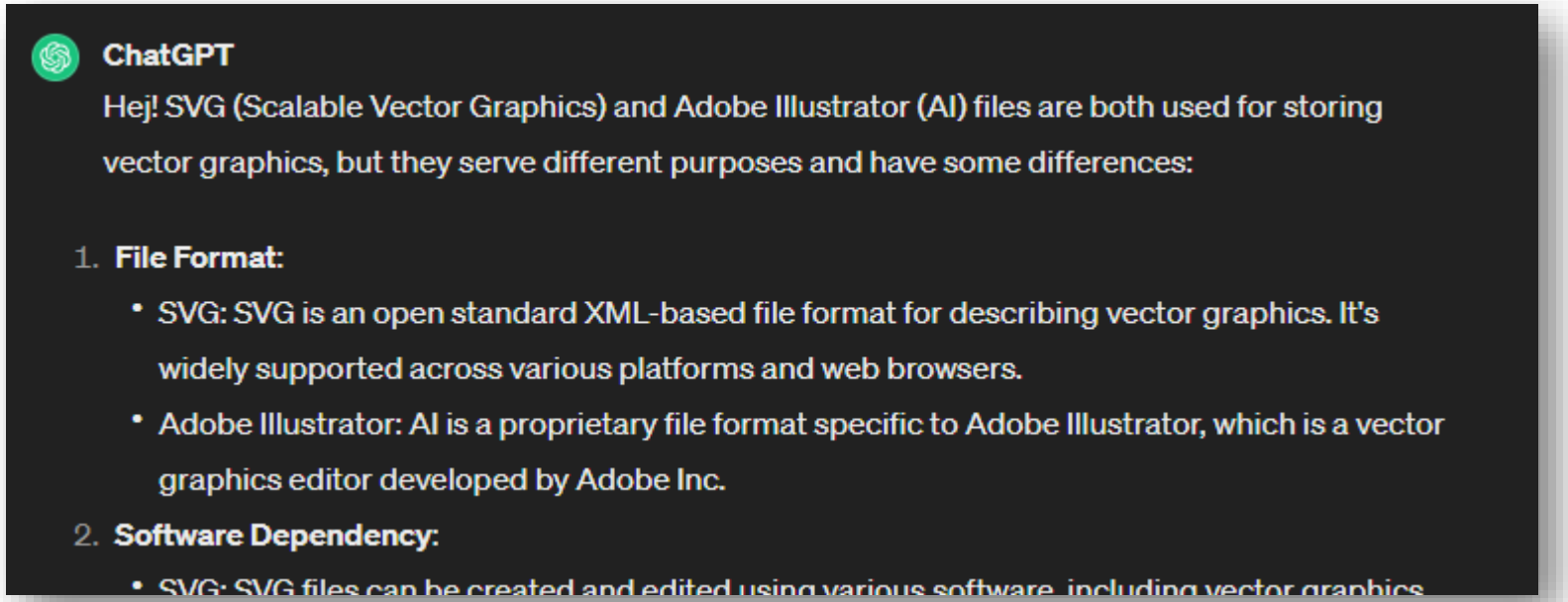
Details:



3) SVG and Adobe Illustrator

JÖNKÖPING UNIVERSITY

- → Adobe Illustrator is a tool to create vectlor graphics
- → However, there are differences between SVG (.svg) and AI (.ai) formats



ChatGPT

Hej! SVG (Scalable Vector Graphics) and Adobe Illustrator (AI) files are both used for storing vector graphics, but they serve different purposes and have some differences:

1. **File Format:**
   - SVG: SVG is an open standard XML-based file format for describing vector graphics. It's widely supported across various platforms and web browsers.
   - Adobe Illustrator: AI is a proprietary file format specific to Adobe Illustrator, which is a vector graphics editor developed by Adobe Inc.
2. **Software Dependency:**
   - SVG: SVG files can be created and edited using various software, including vector graphics

Source: ChatGPT, https://chat.openai.com/

3) SVG and Adobe Illustrator

JÖNKÖPING UNIVERSITY

- Open standard
- Created and edited through many softwares (inkscape, svgator, vecta.io, svectr, svgMorpheus, chartist…)
- Basic shapes, text, paths, gradients, and basic effects
- Supports interactivity and animation through JavaScript and CSS
- Highly compatible with web technologies and are often used for web graphics, icons, logos, and simple illustrations

- Proprietary format from Adobe
- AI files are used with Adobe Illustrator
- Advanced drawing tools, effects, typography, color management, and more complex vector graphics
- AI file format itself doesn't inherently support interactivity or animation
- Used within the Adobe ecosystem and are commonly exchanged between designers and printers for professional graphic design projects

Source: ChatGPT, https://chat.openai.com/

4) SVG basics

JÖNKÖPING UNIVERSITY

→ To create SVG files, you need:

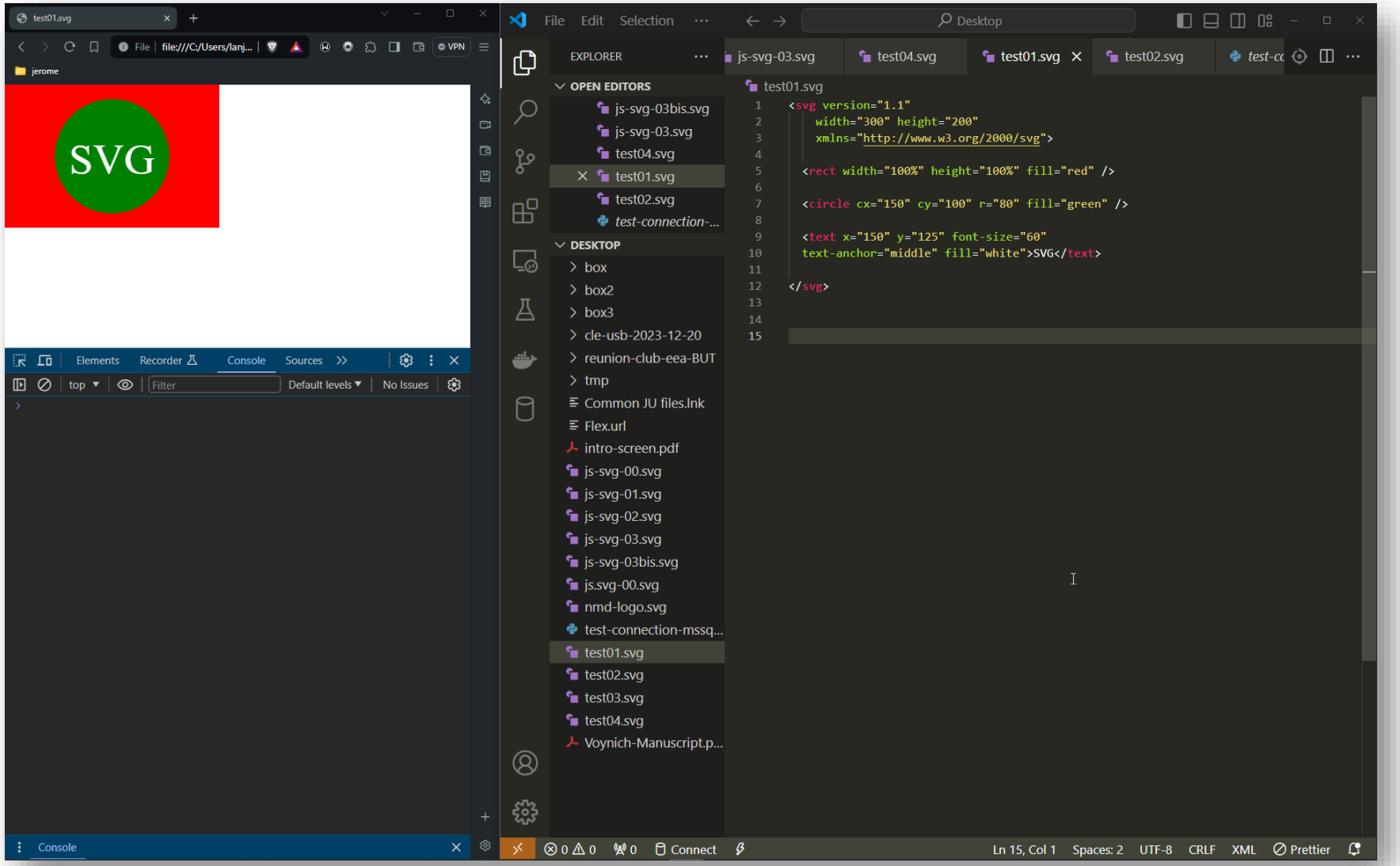
- a SVG editor (inkscape) or Adobe Illustrator and then export to SVG

OR

- A text editor (Visual Studio Code, Notepad++, …) and a browser to see the result

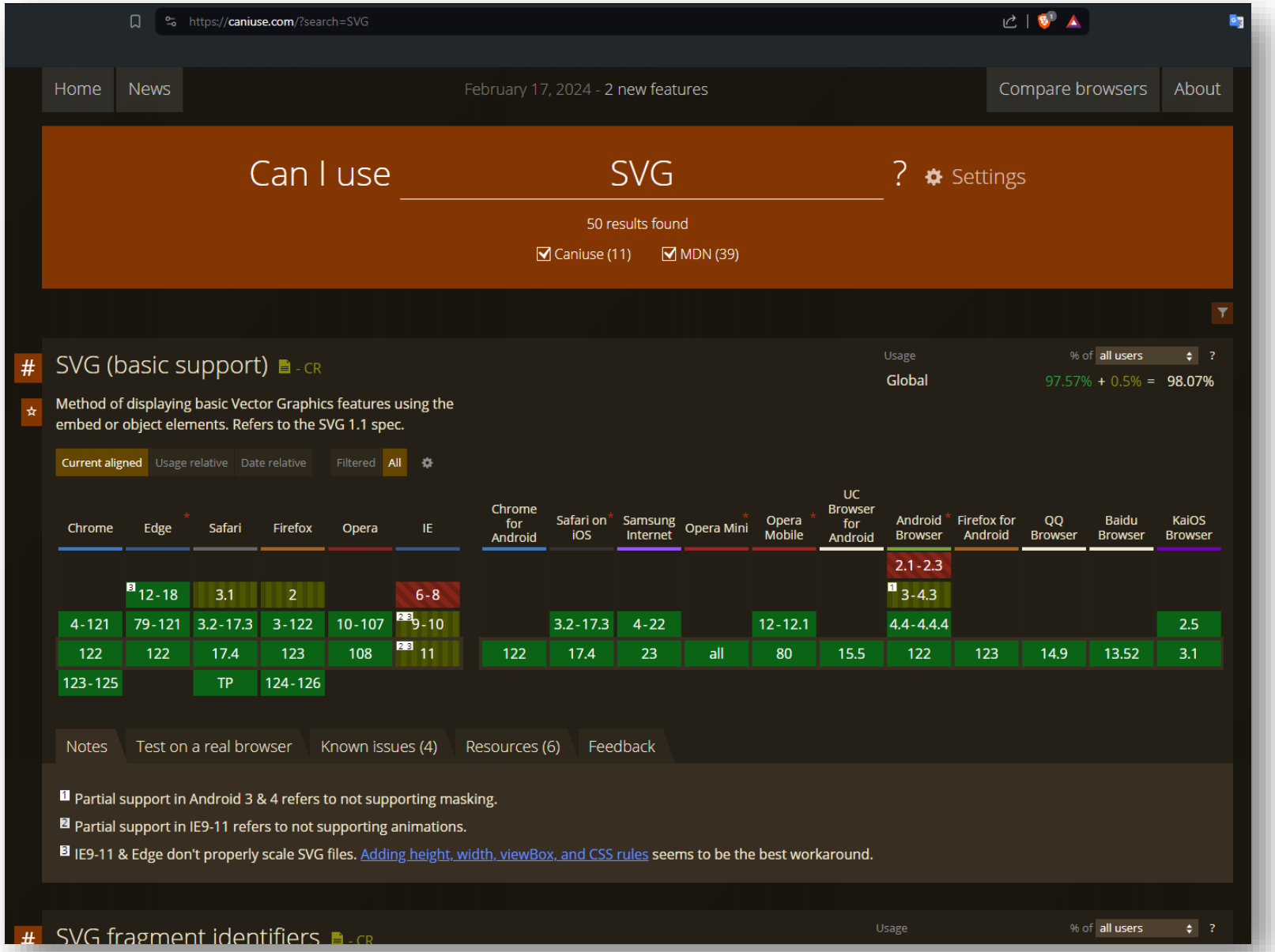
4) SVG basics

- Screen organization when developing SVG files
- Browser on the left and Code Editor on the right



4) SVG basics

- Is my browser compatible with SVG?
- Source: https://caniuse.com/?search=SVG



4) SVG basics

JÖNKÖPING UNIVERSITY

```
<svg version="1.1"
     width="300" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <rect width="100%" height="100%" fill="darkviolet" />

  <text x="150" y="125" font-size="80" font-weight="bold"
            text-anchor="middle" fill="white">
    NMD
  </text>

</svg>
```




```svg
<svg version="1.1"
     width="300" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <rect width="100%" height="100%" fill="darkviolet" />

  <text x="150" y="125" font-size="80" font-weight="bold"
    text-anchor="middle" fill="white">
    NMD
  </text>

</svg>
```

4) SVG basics

JÖNKÖPING UNIVERSITY

```
<svg version="1.1"
     width="300" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <rect width="100%" height="100%" fill="darkviolet" />

  <text x="150" y="125" font-size="80" font-weight="bold"
            text-anchor="middle" fill="white">
    NMD
  </text>

</svg>
```



?

TAGS
Opening TAGS
Closing TAGS
Self-closing TAGS

4) SVG basics

JÖNKÖPING UNIVERSITY

```
<svg version="1.1"
     width="300" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <rect width="100%" height="100%" fill="darkviolet" />

  <text x="150" y="125" font-size="80" font-weight="bold"
            text-anchor="middle" fill="white">
    NMD
  </text>

</svg>
```



?

properties
values
property=”value”

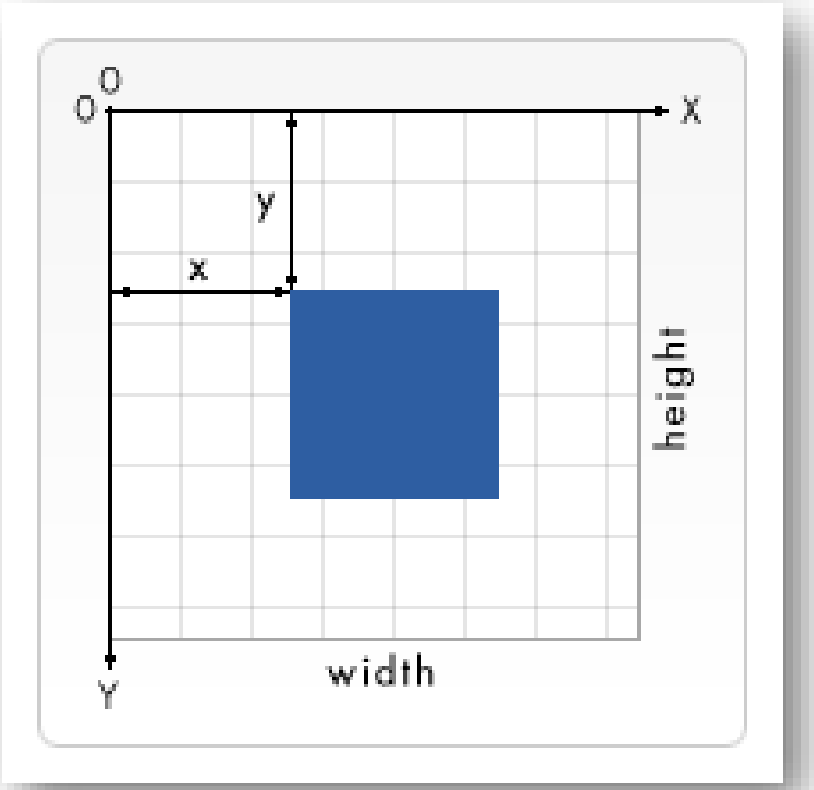
4) SVG basics

JÖNKÖPING UNIVERSITY

→ Positions

```
<rect x="0" y="0" width="100" height="100" />
```

```
<svg width="200" height="200" viewBox="0 0 100 100">…</svg>
```



Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Positions

4) SVG basics

JÖNKÖPING UNIVERSITY

→ Basic shapes

```
<?xml version="1.0" standalone="no"?>
<svg width="200" height="250" version="1.1" xmlns="http://www.w3.org/2000/svg">

  <rect x="10" y="10" width="30" height="30" stroke="black" fill="transparent" stroke-
width="5"/>
  <rect x="60" y="10" rx="10" ry="10" width="30" height="30" stroke="black" fill="transparent"
stroke-width="5"/>

  <circle cx="25" cy="75" r="20" stroke="red" fill="transparent" stroke-width="5"/>
  <ellipse cx="75" cy="75" rx="20" ry="5" stroke="red" fill="transparent" stroke-width="5"/>

  <line x1="10" x2="50" y1="110" y2="150" stroke="orange" stroke-width="5"/>
  <polyline points="60 110 65 120 70 115 75 130 80 125 85 140 90 135 95 150 100 145"
      stroke="orange" fill="transparent" stroke-width="5"/>

  <polygon points="50 160 55 180 70 180 60 190 65 205 50 195 35 205 40 190 30 180 45 180"
      stroke="green" fill="transparent" stroke-width="5"/>

  <path d="M20,230 Q40,205 50,230 T90,230" fill="none" stroke="blue" stroke-width="5"/>
</svg>
```

Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Basic_Shapes

4) SVG basics

JÖNKÖPING UNIVERSITY

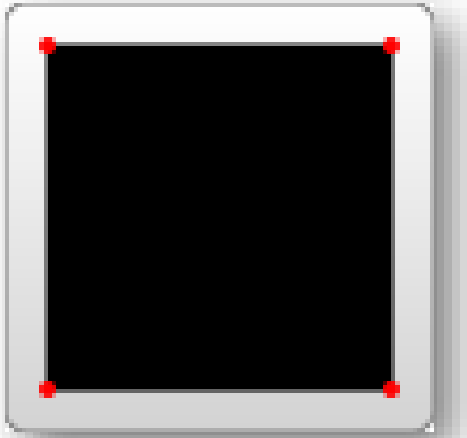
→ Paths

```
<svg width="100" height="100" xmlns="http://www.w3.org/2000/svg">

  <path d="M 10 10 H 90 V 90 H 10 L 10 10"/>

  <!-- Points -->
  <circle cx="10" cy="10" r="2" fill="red"/>
  <circle cx="90" cy="90" r="2" fill="red"/>
  <circle cx="90" cy="10" r="2" fill="red"/>
  <circle cx="10" cy="90" r="2" fill="red"/>

</svg>
```



Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Paths

4) SVG basics

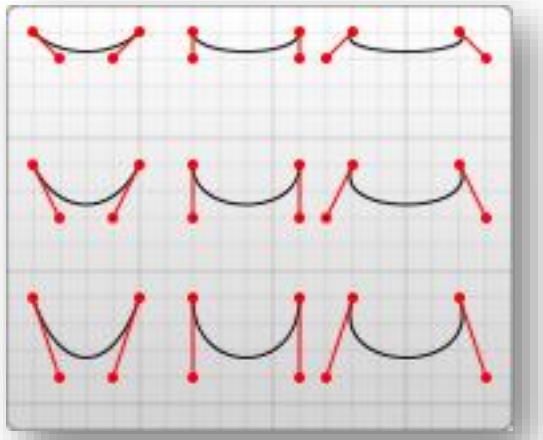
JÖNKÖPING UNIVERSITY

→ Paths

```
<svg width="190" height="160" xmlns="http://www.w3.org/2000/svg">

  <path d="M 10 10 C 20 20, 40 20, 50 10" stroke="black" fill="transparent"/>
  <path d="M 70 10 C 70 20, 110 20, 110 10" stroke="black" fill="transparent"/>
  <path d="M 130 10 C 120 20, 180 20, 170 10" stroke="black" fill="transparent"/>
  <path d="M 10 60 C 20 80, 40 80, 50 60" stroke="black" fill="transparent"/>
  <path d="M 70 60 C 70 80, 110 80, 110 60" stroke="black" fill="transparent"/>
  <path d="M 130 60 C 120 80, 180 80, 170 60" stroke="black" fill="transparent"/>
  <path d="M 10 110 C 20 140, 40 140, 50 110" stroke="black" fill="transparent"/>
  <path d="M 70 110 C 70 140, 110 140, 110 110" stroke="black" fill="transparent"/>
  <path d="M 130 110 C 120 140, 180 140, 170 110" stroke="black" fill="transparent"/>

</svg>
```



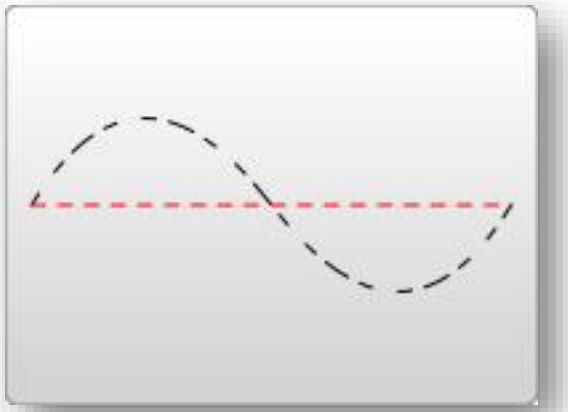
Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Paths

4) SVG basics

JÖNKÖPING UNIVERSITY

→ Fills and Stroke

```
<?xml version="1.0" standalone="no"?>
<svg width="200" height="150" xmlns="http://www.w3.org/2000/svg" version="1.1">
  <path d="M 10 75 Q 50 10 100 75 T 190 75" stroke="black"
    stroke-linecap="round" stroke-dasharray="5,10,5" fill="none"/>
  <path d="M 10 75 L 190 75" stroke="red"
    stroke-linecap="round" stroke-width="1" stroke-dasharray="5,5" fill="none"/>
</svg>
```



Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Paths

4) SVG basics

JÖNKÖPING UNIVERSITY

→ Fills and Stroke **with CSS**!

```
<rect x="10" height="180" y="10" width="180" style="stroke: black; fill: red;"/>
```

```
<?xml version="1.0" standalone="no"?>
<svg width="200" height="200" xmlns="http://www.w3.org/2000/svg" version="1.1">
  <defs>
    <style><![CDATA[
       #MyRect {
         stroke: black;
         fill: red;
       }
    ]]></style>
  </defs>
  <rect x="10" height="180" y="10" width="180" id="MyRect"/>
</svg>
```

Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Fills_and_Strokes

4) SVG basics

JÖNKÖPING UNIVERSITY

→ Fills and Stroke **with CSS style files**!

CSS

```css
#MyRect {
  fill: red;
  stroke: black;
}
```

CSS

```css
#MyRect:hover {
  stroke: black;
  fill: blue;
}
```

```xml
<?xml version="1.0" standalone="no"?>
<?xml-stylesheet type="text/css" href="style.css"?>

<svg width="200" height="150" xmlns="http://www.w3.org/2000/svg" version="1.1">
  <rect height="10" width="10" id="MyRect"/>
</svg>
```

Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Fills_and_Strokes

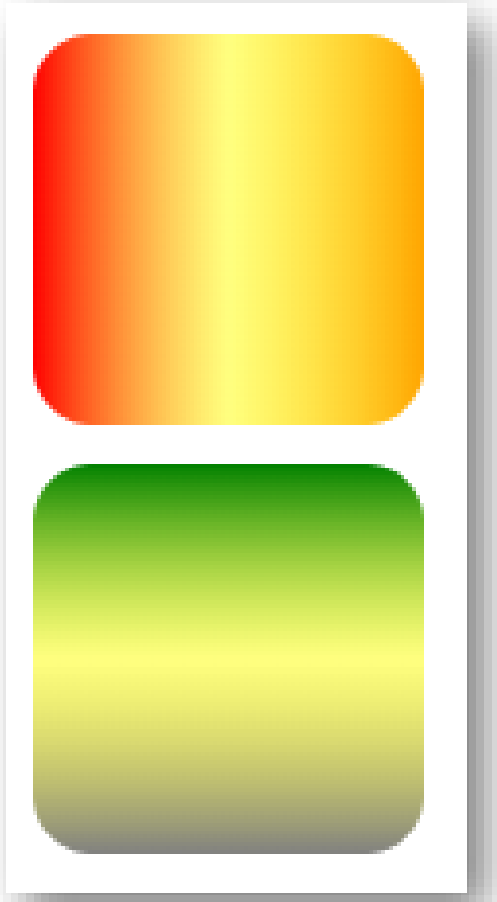
4) SVG basics

JÖNKÖPING UNIVERSITY

→ Gradients

```svg
<svg width="120" height="240" version="1.1" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="Gradient1">
      <stop class="stop1" offset="0%" />
      <stop class="stop2" offset="50%" />
      <stop class="stop3" offset="100%" />
    </linearGradient>
    <linearGradient id="Gradient2" x1="0" x2="0" y1="0" y2="1">
      <stop offset="0%" stop-color="green" />
      <stop offset="50%" stop-color="yellow" stop-opacity="0.5" />
      <stop offset="100%" stop-color="gray" />
    </linearGradient>
  </defs>
  <style>
    #rect1 { fill: url(#Gradient1); }
    .stop1 { stop-color: red; }
    .stop2 { stop-color: yellow; stop-opacity: 0.5; }
    .stop3 { stop-color: orange; }
  </style>

  <rect id="rect1" x="10" y="10" rx="15" ry="15" width="100" height="100" />
  <rect x="10" y="120" rx="15" ry="15" width="100" height="100" fill="url(#Gradient2)" />
</svg>
```

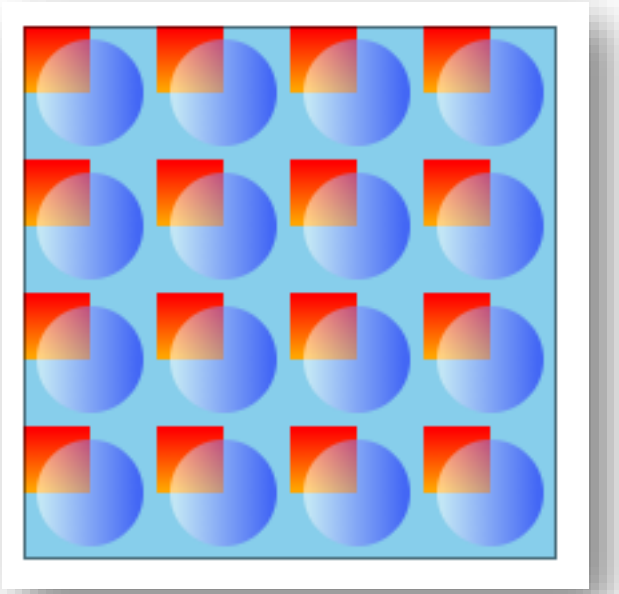


Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Gradients

4) SVG basics

JÖNKÖPING UNIVERSITY

→ Patterns



```svg
<svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="Gradient1">
      <stop offset="5%" stop-color="white" />
      <stop offset="95%" stop-color="blue" />
    </linearGradient>
    <linearGradient id="Gradient2" x1="0" x2="0" y1="0" y2="1">
      <stop offset="5%" stop-color="red" />
      <stop offset="95%" stop-color="orange" />
    </linearGradient>

    <pattern id="Pattern" x="0" y="0" width=".25" height=".25">
      <rect x="0" y="0" width="50" height="50" fill="skyblue" />
      <rect x="0" y="0" width="25" height="25" fill="url(#Gradient2)" />
      <circle
        cx="25"
        cy="25"
        r="20"
        fill="url(#Gradient1)"
        fill-opacity="0.5" />
    </pattern>
  </defs>

  <rect fill="url(#Pattern)" stroke="black" width="200" height="200" />
</svg>
```

Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Patterns

4) SVG basics

JÖNKÖPING UNIVERSITY

→ Texts

```
<text x="10" y="10">Hello World!</text>
```

Hello World!

texts.svg

```
<?xml version="1.0" standalone="no"?>
<?xml-stylesheet type="text/css" href="styles-svg.css"?>

<svg width="300" height="200" xmlns="http://www.w3.org/2000/svg">
  <text x="20" y="20">
    Hello World!
  </text>
  <text x="10" y="40">
    This is
    <tspan font-weight="bold" fill="red"> a script </tspan>
    font!
  </text>
</svg>
```

styles-svg.css > ...

```
@import url('https://fonts.googleapis.com/css2?family=Courgette&family=Devonshi

text {
    font: 24px "Courgette", Helvetica, Arial, sans-serif;
}

tspan {
    font: 24px "Devonshire", sans-serif;
}
```

Hello World!
This is a script font!

X 300% on the SVG image





X 300% on the bitmap image

Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Texts

4) SVG basics

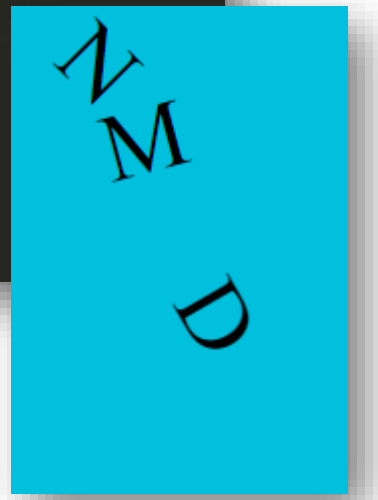
JÖNKÖPING UNIVERSITY

→ Basic transformations

basic-transforms.svg

```svg
<?xml version="1.0" standalone="no"?>

<svg width="300" height="200" xmlns="http://www.w3.org/2000/svg" style="background-color: #00C0DE">
  <text x="0" y="0" transform="translate(10,10) rotate(45)">
    N
  </text>
  <text x="0" y="0" transform="translate(20,30) rotate(-15)">
    M
  </text>
  <text x="0" y="0" transform="translate(30,50) rotate(60)">
    D
  </text>
</svg>
```



Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Basic_Transformations

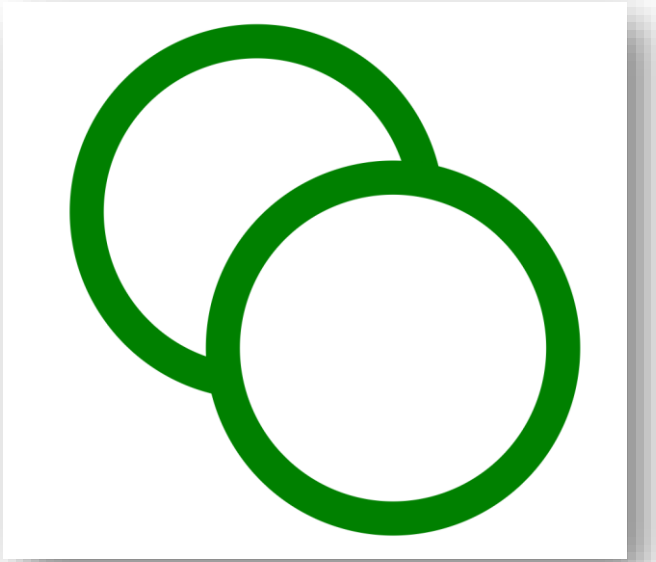
4) SVG basics

JÖNKÖPING UNIVERSITY

→ Grouping

grouping.svg

```svg
<svg viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">
  <!-- Using g to inherit presentation attributes -->
  <g fill="white" stroke="green" stroke-width="5">
    <circle cx="40" cy="40" r="25" />
    <circle cx="60" cy="60" r="25" />
  </g>
</svg>
```



Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Element/g

4) SVG basics

JÖNKÖPING UNIVERSITY

→ Images

images-05.svg

```svg
<svg
  version="1.1"
  xmlns="http://www.w3.org/2000/svg"
  xmlns:xlink="http://www.w3.org/1999/xlink"
  width="600"
  height="400">
  <image
    x="10"
    y="60"
    width="400"
    href="ju.jpg" />
</svg>
```



Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Other_content_in_SVG

4) SVG basics

JÖNKÖPING UNIVERSITY

→ Clipping and Masking



images-03.svg

```svg
<svg
  version="1.1"
  xmlns="http://www.w3.org/2000/svg"
  xmlns:xlink="http://www.w3.org/1999/xlink"
  width="400"
  height="400" style="background-color: lightblue;">
   <defs>
       <clipPath id="clip">
           <circle cx="100" cy="100" r="100" fill="black" />
       </clipPath>

    </defs>
    <image x="-100" y="-100" width="400" height="400" href="doc-tikz.png" clip-path="url(#clip)" />

</svg>
```

Source: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Clipping_and_masking

5) SVG and Javascript

JÖNKÖPING UNIVERSITY

- → **Javascript (JS)** is a very important programming language both on the client and the server side
- → It is an **asynchronous event-driven**, **object-oriented**, **dynamic typing language**

js-svg-00.svg

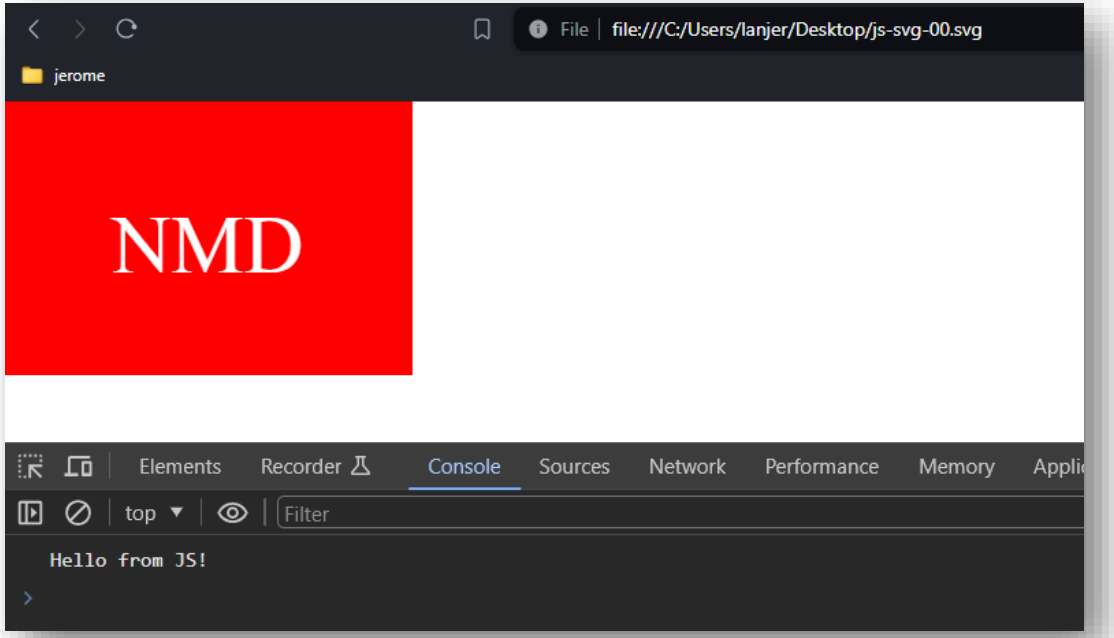
C: > Users > lanjer > Desktop > js-svg-00.svg

```svg
<svg version="1.1"
     width="300" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <rect width="100%" height="100%" fill="red" />

  <text x="150" y="125" font-size="60"
  text-anchor="middle" fill="white">NMD</text>

  <script>
  <![CDATA[
      //put your JS code here!
      console.log("Hello from JS!")
  ]]>
  </script>
</svg>
```



5) SVG and Javascript

JÖNKÖPING UNIVERSITY

```svg
<svg version="1.1"
     width="300" height="200"
     xmlns="http://www.w3.org/2000/svg">

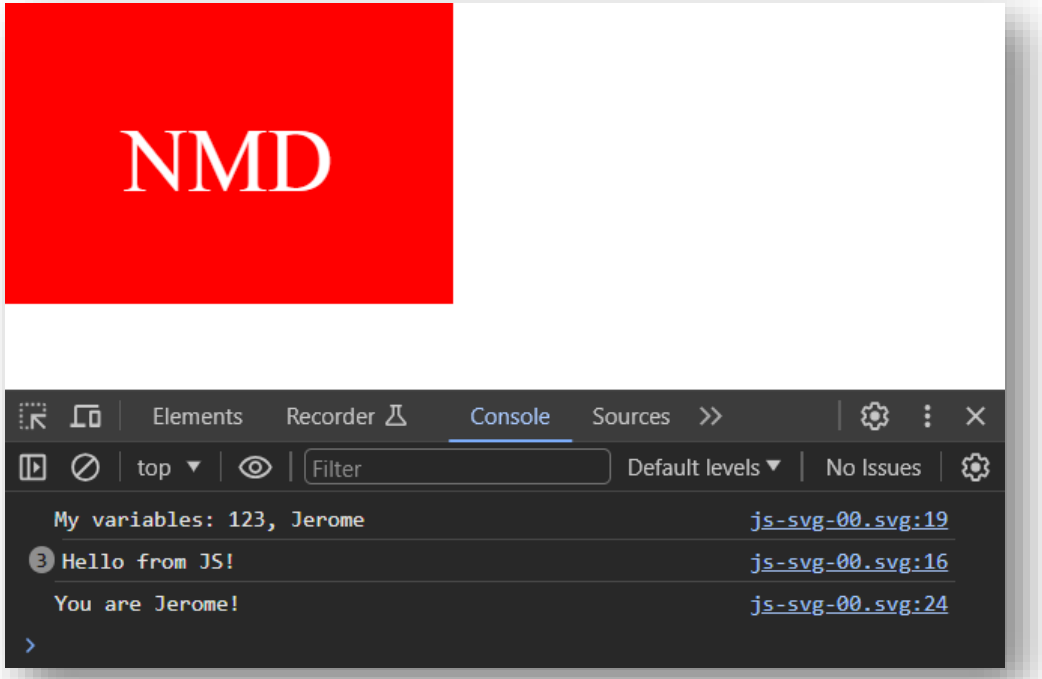
  <rect width="100%" height="100%" fill="red" />

  <text x="150" y="125" font-size="60"
  text-anchor="middle" fill="white">NMD</text>

  <script>
  <![CDATA[
      //put your JS code here!
      var myNumber=123
      var myString="Jerome"
      function sayHello(yourMessage) {
        console.log(yourMessage)
      }

      console.log("My variables: "+myNumber+", "+myString)
      for (var i=0; i<3; i++) {
        sayHello("Hello from JS!")
      }
      if (myString=="Jerome") {
        console.log("You are Jerome!")
      } else {
        console.log("You are not Jerome!")
      }

  ]]>
  </script>
</svg>
```



5) SVG and Javascript

JÖNKÖPING UNIVERSITY

- → **Javascript** is a very important programming language both on the client and the server side
- → It is an **asynchronous event-driven**, **object-oriented**, **dynamic typing language**

```
<script>
<![CDATA[
    //put your JS code here!
    console.log("Hello from JS!")
]]>
</script>
```

- → Why do you need?

<![CDATA[

…

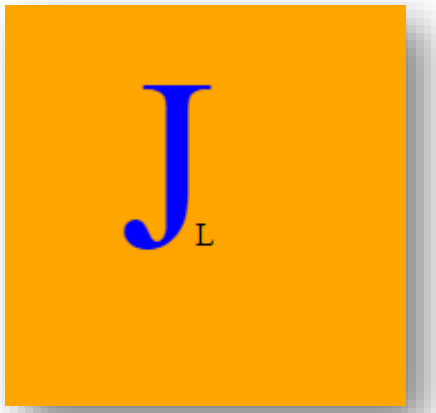
]]>

- → Because in the <script>…</script>, you can have “<“ or “>” characters which will collide with your SVG tags!

5) SVG and Javascript

JÖNKÖPING UNIVERSITY

→ Accessing and changing the **properties** of SVG objects with **CSS**



js-svg-03.svg

```svg
<svg version="1.1"
     width="200" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <defs>
    <style>
      #j {
        font-size: 120px;
        fill: blue;
      }
    </style>
  </defs>
  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />

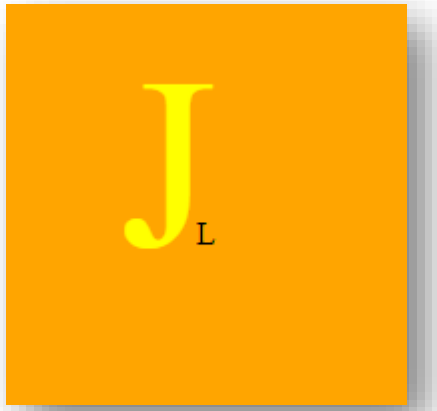
  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120" text-anchor="middle">L</text>

</svg>
```

5) SVG and Javascript

JÖNKÖPING UNIVERSITY

→ Accessing and changing the **properties** of SVG objects with **JS**



js-svg-03bis.svg

```svg
<svg version="1.1"
     width="200" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <defs>
    <style>
      #j {
        font-size: 120px;
        fill: black;
      }
    </style>
  </defs>
  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />

  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120"  text-anchor="middle">L</text>

  <script>
  <![CDATA[

    var j = document.getElementById("j")
    j.style.fill = "yellow"

  ]]>
  </script>
</svg>
```

6) SVG Events

JÖNKÖPING UNIVERSITY

- JS is **asynchronous**
  - If you want to use JS to change SVG files, you MUST create synchronized events using the **Event functions** that exist in JS
- JS can **interact** with objects
  - You can access any object in your SVG file **using an id** (like in the HTML DOM)!
- To **change properties** with JS, you can **use a function** that will be **called** at a certain **frequency** and change the properties of objects inside this function, easy! Let's do it!

6) SVG Events

JÖNKÖPING UNIVERSITY

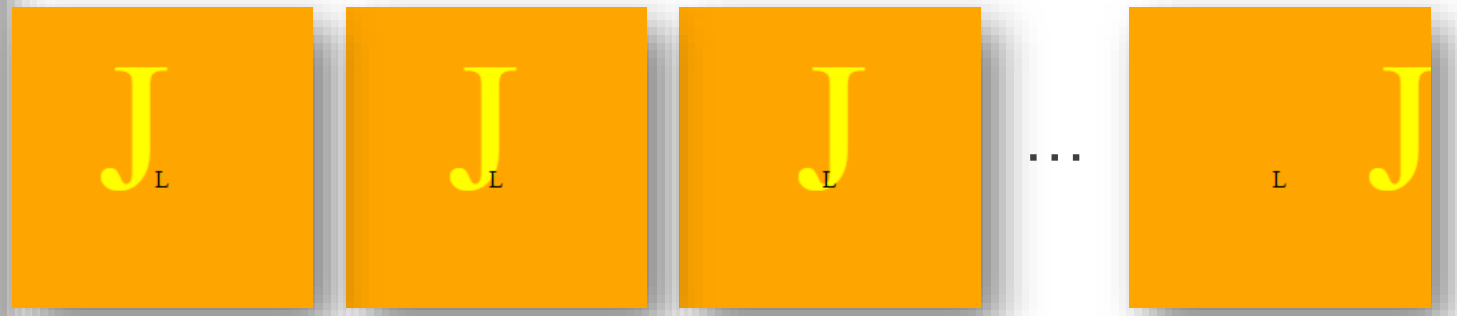
js-svg-04.svg

```svg
<svg version="1.1"
     width="200" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <defs>
    <style>
      #j {
        font-size: 120px;
        fill: blue;
      }
    </style>
  </defs>
  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />

  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120" text-anchor="middle">L</text>


  <script>
    <![CDATA[
      var j = document.getElementById("j")
      j.style.fill = "yellow"
      var moveJ=10
      var timeIntervalForJ = setInterval(function () {
        positionX=j.getAttribute("x")*1
        j.setAttribute("x", positionX+moveJ)
      }, 1000)
    ]]>
  </script>
</svg>
```



6) SVG Events

js-svg-05-right.svg

```svg
<svg version="1.1"
     width="200" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <defs>
    <style>
      #j {
        font-size: 120px;
        fill: blue;
      }
    </style>
  </defs>
  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />

  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120" text-anchor="middle">L</text>


  <script>
    <![CDATA[
      var j = document.getElementById("j")
      j.style.fill = "yellow"
      var moveJ = 10
      var timeIntervalForJ = setInterval(function () {
        positionX = j.getAttribute("x")*1
        j.setAttribute("x", positionX+moveJ)
        if (positionX+moveJ>200) {
          moveJ = -1 * moveJ
        }
      }, 1000)
    ]]>
  </script>
</svg>
```

JÖNKÖPING UNIVERSITY

6) SVG Events

JÖNKÖPING UNIVERSITY

js-svg-06-right-left.svg

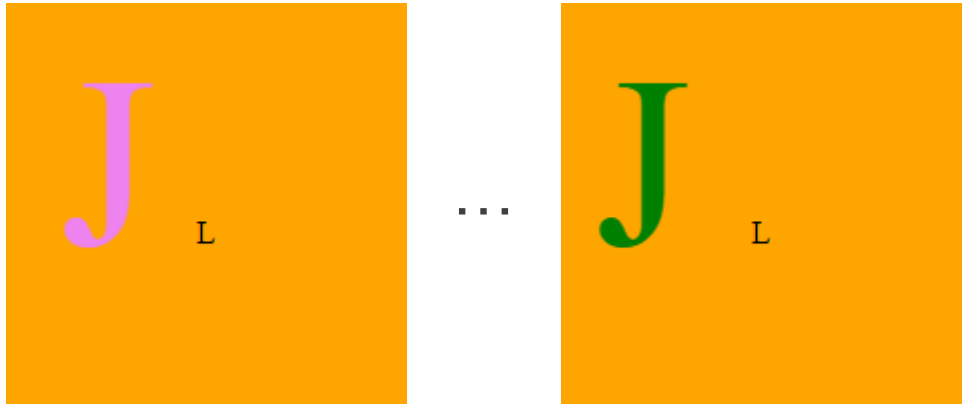
```svg
<svg version="1.1"
     width="200" height="200"
     xmlns="http://www.w3.org/2000/svg">

  <defs>
    <style>
       #j {
         font-size: 120px;
         fill: blue;
       }
    </style>
  </defs>
  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />

  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120" text-anchor="middle">L</text>


  <script>
    <![CDATA[
     var j = document.getElementById("j")
     j.style.fill = "yellow"
     var moveJ = 10
     var timeIntervalForJ = setInterval(function () {
       positionX = j.getAttribute("x")*1
       j.setAttribute("x", positionX+moveJ)
       if ((positionX+moveJ>200) || (positionX+moveJ<0)) {
         moveJ = -1 * moveJ
       }
     }, 100)
    ]]>
  </script>
</svg>
```

6) SVG Events



js-svg-06-right-left-color.svg

```svg
<svg version="1.1"
     width="200" height="200"
     xmlns="http://www.w3.org/2000/svg">

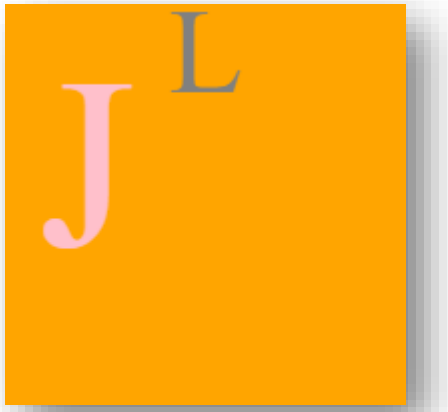
  <defs>
    <style>
      #j {
        font-size: 120px;
        fill: blue;
      }
    </style>
  </defs>
  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />

  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120" text-anchor="middle">L</text>


  <script>
    <![CDATA[
      var j = document.getElementById("j")
      var listOfColors = ["red", "green", "blue", "yellow", "violet", "pink"]
      var colorNumber = 0
      var moveJ = 10
      var timeIntervalForJ = setInterval(function () {
        positionX = j.getAttribute("x")*1
        j.setAttribute("x", positionX+moveJ)
        j.style.fill = listOfColors[colorNumber%listOfColors.length]
        if ((positionX+moveJ>200) || (positionX+moveJ<0)) {
          moveJ = -1 * moveJ
        }
        colorNumber++
      }, 400)
    ]]>
  </script>
</svg>
```

6) SVG Events

→ Animate more than 1 object: create **as many functions** as wanted to change properties!



js-svg-06-right-left-color-up-down.svg

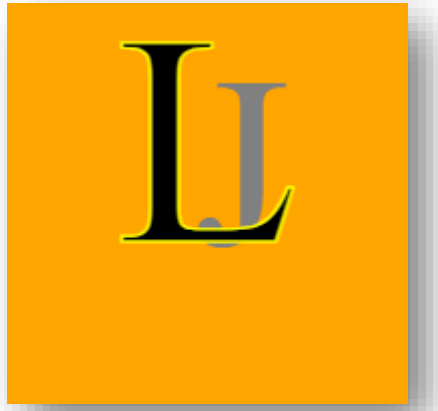
```svg
<svg version="1.1" width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <style>
        #j { font-size: 120px; fill: blue; }
        #l { font-size: 60px; fill: gray; }
    </style>
  </defs>
  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />
  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120" text-anchor="middle" id="l">L</text>
  <script>
    <![CDATA[
      var j = document.getElementById("j")
      var listOfColors = ["red", "green", "blue", "yellow", "violet", "pink"]
      var colorNumber = 0
      var moveJ = 10
      var timeIntervalForJ = setInterval(function () {
        positionX = j.getAttribute("x")*1
        j.setAttribute("x", positionX+moveJ)
        j.style.fill = listOfColors[colorNumber%listOfColors.length]
        if ((positionX+moveJ>200) || (positionX+moveJ<0)) {
          moveJ = -1 * moveJ
        }
        colorNumber++
      }, 400)

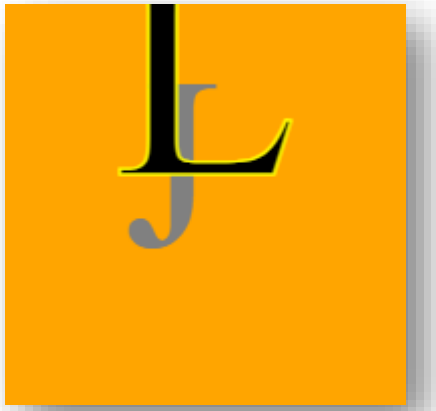
      var l = document.getElementById("l")
      var moveL = 4
      var timeIntervalForL = setInterval(function () {
        positionY = l.getAttribute("y")*1
        l.setAttribute("y", positionY+moveL)
        if ((positionY+moveL>200) || (positionY+moveL<0)) {
          moveL = -1 * moveL
        }
      }, 200)

    ]]>
  </script>
</svg>
```

6) SVG Events

→ If you dislike ”if” tests, just use **Maths**!





js-svg-07-math.svg

```svg
<svg version="1.1" width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />
  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120" text-anchor="middle" id="l">L</text>
  <script>
    <![CDATA[
      var j = document.getElementById("j")
      j.setAttribute("font-size", 120)
      j.setAttribute("fill", "gray")
      var moveNumber = 0
      var timeIntervalForJ = setInterval(function () {
        positionX = 20+Math.abs(Math.sin(moveNumber*2*Math.PI/360)*160)
        j.setAttribute("x", positionX)
        moveNumber++
      }, 100)

      var l = document.getElementById("l")
      l.setAttribute("font-size", 150)
      l.setAttribute("stroke", "yellow")
      var timeIntervalForL = setInterval(function () {
        positionY = 20+Math.abs(Math.sin(moveNumber*2*Math.PI/360)*160)
        l.setAttribute("y", positionY)
      }, 200)

    ]]>
  </script>
</svg>
```

6) SVG Events

→ You can add ”click” **events** to your code and react to them!



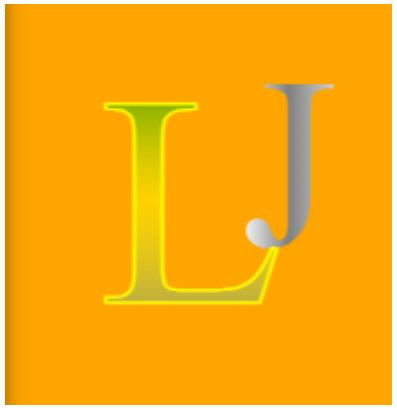
js-svg-09-math.svg

```svg
<svg version="1.1" width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="Gradient1">
      <stop class="stop1" offset="0%" stop-color="white"/>
      <stop class="stop2" offset="50%" stop-color="gray"/>
      <stop class="stop3" offset="100%" stop-color="lightgray"/>
    </linearGradient>
    <linearGradient id="Gradient2" x1="0" x2="0" y1="0" y2="1">
      <stop offset="0%" stop-color="green" />
      <stop offset="50%" stop-color="yellow" stop-opacity="0.5" />
      <stop offset="100%" stop-color="gray" />
    </linearGradient>
  </defs>

  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />
  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120" text-anchor="middle" id="l">L</text>
  <script>
    <![CDATA[
      var j = document.getElementById("j")
      j.setAttribute("font-size", 120)
      j.setAttribute("fill", "url(#Gradient1)")
      var moveNumber = 0
      var timeIntervalForJ = setInterval(function () {
        positionX = 20+Math.abs(Math.sin(moveNumber*2*Math.PI/360)*160)
        j.setAttribute("x", positionX)
        moveNumber++
      }, 100)
      var l = document.getElementById("l")
      l.setAttribute("font-size", 150)
      l.setAttribute("stroke", "yellow")
      l.setAttribute("fill", "url(#Gradient2)")
      var timeIntervalForL = setInterval(function () {
        positionY = 20+Math.abs(Math.sin(moveNumber*2*Math.PI/360)*160)
        l.setAttribute("y", positionY)
      }, 200)
      var clicked=0
      document.querySelector("#j").addEventListener("click", (e) => {
        if (clicked%2==0) {
            e.target.style.fill = "darkred"; // "url(#Gradient1)"
        } else {
            e.target.style.fill = "url(#Gradient2)"
        }
        //console.log("click")
        clicked++
      });
    ]]>
  </script>
</svg>
```

6) SVG Events

→ You can add some ”random” values to the coordinates or any attribute using the **Math.random()** JS function




```svg
<svg version="1.1" width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="Gradient1">
      <stop class="stop1" offset="0%" stop-color="white"/>
      <stop class="stop2" offset="50%" stop-color="gray"/>
      <stop class="stop3" offset="100%" stop-color="lightgray"/>
    </linearGradient>
    <linearGradient id="Gradient2" x1="0" x2="0" y1="0" y2="1">
      <stop offset="0%" stop-color="green" />
      <stop offset="50%" stop-color="yellow" stop-opacity="0.5" />
      <stop offset="100%" stop-color="gray" />
    </linearGradient>
  </defs>

  <!-- BACKGROUND COLOR -->
  <rect width="100%" height="100%" fill="orange" />
  <text x="80" y="120" text-anchor="middle" id="j">J</text>
  <text x="100" y="120" text-anchor="middle" id="l">L</text>
  <script>
    <![CDATA[
      var j = document.getElementById("j")
      j.setAttribute("font-size", 120)
      j.setAttribute("fill", "url(#Gradient1)")
      var moveNumber = 0
      var timeIntervalForJ = setInterval(function () {
        positionX = 20+Math.abs(Math.sin(moveNumber*2*Math.PI/360)*160)+Math.round(Math.random()*4)
        j.setAttribute("x", positionX)
        moveNumber++
      }, 100)
      var l = document.getElementById("l")
      l.setAttribute("font-size", 150)
      l.setAttribute("stroke", "yellow")
      l.setAttribute("fill", "url(#Gradient2)")
      var timeIntervalForL = setInterval(function () {
        positionY = 20+Math.abs(Math.sin(moveNumber*2*Math.PI/360)*160)+Math.round(Math.random()*4)
        l.setAttribute("y", positionY)
      }, 200)
      var clicked=0
      document.querySelector("#j").addEventListener("click", (e) => {
        if (clicked%2==0) {
            e.target.style.fill = "darkred"; // "url(#Gradient1)"
        } else {
            e.target.style.fill = "url(#Gradient2)"
        }
        //console.log("click")
        clicked++
      });
    ]]>
  </script>
</svg>
```

7) Resources

JÖNKÖPING UNIVERSITY

- **General SVG**

  MDN: https://developer.mozilla.org/en-US/docs/Web/SVG, https://developer.mozilla.org/en-US/docs/Learn/HTML/Multimedia_and_embedding/Adding_vector_graphics_to_the_Web, https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Getting_Started

  W3Schools: https://www.w3schools.com/html/html5_svg.asp

- **Interactions Javascript/SVG**

  Including JS in SVG: https://stackoverflow.com/questions/5378559/including-javascript-in-svg

  https://stackoverflow.com/questions/30940609/interacting-with-a-svg-image

  Event hover, mouse: https://www.petercollingridge.co.uk/tutorials/svg/interactive/javascript/, https://www.petercollingridge.co.uk/tutorials/svg/interactive/mouseover-effects/

  Embedding SVG into HTML: https://stackoverflow.com/questions/14068031/embedding-external-svg-in-html-for-javascript-manipulation, https://cravencode.com/post/javascript/embed-svg-with-javascript/, https://vecta.io/blog/best-way-to-embed-svg

  Creating SVG elements using JS: https://www.motiontricks.com/creating-dynamic-svg-elements-with-javascript/

  Animated watch: https://www.freecodecamp.org/news/svg-javascript-tutorial/

  Paths: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Paths

7) Resources

JÖNKÖPING UNIVERSITY

→ **Difference between Adobe Illustrator and SVG format**

https://www.quora.com/What-is-the-difference-between-SVG-and-AI-Adobe-Illustrator-files-in-terms-of-printing

→ **Inline SVG**

Library to work with img as svg in HTML: https://github.com/jonnyhaynes/inline-svg

O'Reilly: https://www.oreilly.com/library/view/svg-essentials-2nd/9781491945308/ch04.html


JÖNKÖPING UNIVERSITY



JÖNKÖPING UNIVERSITY